



西南科技大学

Southwest University of Science and Technology

信息隐藏技术论文

题目名称： 数字图像空域隐写与分析技术的实现

学 院 名 称 计算机科学与技术学院

专 业 名 称

学 生 姓 名

学 号

指 导 教 师 周宏毅

二〇二四年十一月

数字图像空域隐写与分析技术的实现

摘要：

本项目实现了数字图像空域隐写和分析技术，重点在 BMP 图像格式下进行隐写与分析研究。通过最低有效位（LSB）和多位最低有效位（MLSB）隐写方法，将大规模和小规模的秘密数据嵌入图像，并通过 X2 分析、信息量估计法、RS 分析和 GPC 分析等技术对隐写效果进行检测和评估。此外，还研究了抗分析隐写技术，并对隐写图片进行多种分析方法的对比实验。实验结果验证了不同隐写量在各种分析方法下的检测效果，为图像隐写和隐写分析技术提供了实践参考。

关键词：

数字图像；空域隐写；BMP 图像；最低有效位；X2 分析；RS 分析；GPC 分析；信息量估计；抗分析隐写

Implementation of Digital Image Spatial Domain Steganography and Analysis Techniques

Abstract:

This project implements digital image spatial domain steganography and analysis techniques, with a focus on BMP image format for embedding and analysis. By using Least Significant Bit (LSB) and Multiple Least Significant Bit (MLSB) steganography methods, large and small-scale secret data are embedded into images. The effectiveness of the steganography process is evaluated using techniques such as X2 analysis, information estimation, RS analysis, and Group Pair Consistency (GPC) analysis. Additionally, anti-analysis steganography techniques are explored, and multiple analysis methods are compared in experimental tests. The experimental results validate the detection effectiveness of different amounts of hidden data under various analysis methods, providing practical references for image steganography and steganalysis techniques.

Key words:

Steganography; Grammar-based steganography; Data hiding; Information security; File format analysis

目 录

目 录	III
任务一：完成对 BMP 位图格式图像文件的 LSB 顺序隐写和 X^2 分析	5
1.1 LSB 顺序隐写技术	5
1.1.1 LSB 隐写原理	5
1.1.2 实现过程	5
1.1.3 代码	5
1.1.4 实现效果	5
1.2 X^2 分析	7
1.2.1 卡方检验原理	7
1.2.2 实现过程	8
1.2.3 代码	8
1.2.3 实现效果	9
1.3 结论	10
任务二：完成对 BMP 位图格式图像文件的 LSB 和 MLSB 数据位的随机隐写并进行信息量估计法分析和 RS 分析。	11
2.1 LSB 随机隐写技术	11
2.1.1 概述	11
2.1.2 代码	11
2.1.3 实现效果	11
2.2 MLSB 随机隐写技术	12
2.2.1 MLSB 隐写原理	14
2.2.2 代码	14
2.2.3 实现效果	14
2.3 信息量估计法分析	15
2.3.1 原理	17
2.3.2 代码	17
2.3.3 实现效果	17
2.4 RS 分析	20
2.4.1 原理	20
2.4.2 代码	21
2.4.3 实现效果	26
2.5 实验结果与讨论	27
任务三：完成对 BMP 位图格式图像文件的抗分析的 LSB 数据位的随机隐写和分析	33
3.1 抗 RS 分析代码	33
3.2 分析	36
3.2.1 肉眼观察	错误!未定义书签。
3.2.2 卡方分析	36
3.2.3 信息量估计分析	37
3.2.4 RS 分析	38

3.3 结论	41
结论	43
参考文献	44
附录	45

任务一：完成对 BMP 位图格式图像文件的 LSB 顺序隐写和 X^2 分析

1.1 LSB 顺序隐写技术

1.1.1 LSB 隐写原理

LSB (Least Significant Bit) 即最低有效位，是图像数据存储中的最低位。以 RGB 图像为例，每个像素包含三个通道（红、绿、蓝），每个通道的颜色值通常由 8 位（0-255）表示。LSB 隐写利用了人眼对颜色变化的不敏感性，将秘密信息嵌入到这些最低位上。由于人眼对色彩变化不敏感，嵌入的信息不会对图像的视觉质量产生明显影响，因此实现了隐蔽的数据传输。

1.1.2 实现过程

1. 图像读取与数据转换：使用 Pillow 库打开指定路径的图像文件，以只读模式打开秘密信息文件，并读取其内容。
2. 秘密文件读取与转换：通过 `file.read()` 读取文件内容，并将每个字符转换为 8 位的二进制字符串，根据嵌入率得到秘密信息。
3. 通过两层循环遍历图像的宽度和高度，对每个像素进行操作，获取每个像素的 RGB 值。对于每个颜色通道（R、G、B），将其最低有效位（LSB）替换为秘密信息的二进制位。
4. 图像保存：完成隐写后，使用 `putpixel` 方法将修改后的像素值写回图像。

1.1.3 代码

```
def perform_lsb_steganography(image_path, output_path, embedding_rate,
secret_file_path):
    # 打开灰度图像文件
    im = Image.open(image_path).convert('L')
    width, height = im.size
    print(width, height)

    # 从文件读取秘密信息并调整嵌入量
    secret_data = read_secret_from_file(secret_file_path, embedding_rate,
width, height, 1)
    secret_length = len(secret_data)
    count = 0 # 用于跟踪已嵌入的比特数
```

```
for h in range(height):
    for w in range(width):
        # 逐像素操作，获取灰度像素值
        pixel = im.getpixel((w, h))

        # 处理灰度像素
        if count < secret_length:
            pixel = (pixel & ~1) | int(secret_data[count]) # 清除最低有效位
并嵌入比特
            count += 1

        # 将修改后的像素写回
        im.putpixel((w, h), pixel)

        # 如果嵌入完成，则退出循环
        if count >= secret_length:
            break
    if count >= secret_length:
        break
```

1.1.4 实现效果

通过隐写不同大小的秘密数据到图像中，我们肉眼可以观察到图像的外观变化极小。

五论是对于大数据隐写还是对小数据隐写，图像的变化都几乎没有。



Figure 1 左图为原图，右图为嵌入率 1 隐写后的图片



Figure 2 左图为原图，右图为嵌入率 0.2 隐写后的图片

1.2 X^2 分析

1.2.1 卡方检验原理

卡方检验 (Chi-Square Test) 是一种用于检验观察频数与理论频数之间差异的统计方法。在 LSB 隐写这种情况下，信息的嵌入方式主要影响图像的最低有效位，从而导致图像的灰度值发生变化，这些变化主要表现为：

- 灰度值的相邻像素对变得更加接近。例如，一个偶数值的像素可能变成相邻的奇数值，或反之。
- 由于这些变动，原本相差较大的灰度值可能变得近似相等。尽管如此，这些变

化并不影响每组灰度值的像素总数。

为了定量分析图像灰度分布的变化，我们使用卡方检验来评估实际观察值与期望值之间的差异。

1.2.2 实现过程

1. 计算灰度直方图：通过统计每个灰度级别（从 0 到 255）的像素数量来完成的。
2. 期望频率的计算：理想的直方图应该是均匀分布的，即每个灰度级别的出现概率是相同的，我们通过计算每个直方图区间中的平均值得到期望频率。
3. 卡方统计量：我们计算载体图像和隐写图像的直方图之间的卡方统计量 χ^2 ，衡量了观察到的频率和期望频率之间的差异，计算自由度的方式是通过遍历灰度值对并且在每个灰度对上进行卡方值的累加。公式如下：

$$\chi^2 = \sum_{i=0}^{127} \frac{(h_i + h_{i+1} - 2h_0)^2}{h_0}$$

4. 使用卡方分布的累积分布函数（CDF）来计算 P 值。P 值越小，隐写的概率越高。

1.2.3 代码

```
def stgPrb(matrix):
    count = np.zeros(256, dtype=int)
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            count[matrix[i, j]] += 1
    h2i = count[0::2]
    h2is = (h2i + count[1::2]) / 2
    filter = h2is != 0
    idx = np.where(filter)[0] # 获取非零值的索引
    r = np.sum((h2i[idx] - h2is[idx]) ** 2) / h2is[idx]
    k = len(idx)
    p = 1 - chi2.cdf(r, k - 1)
    return r, p

def detect_lsb_steganography(carrier_image_path, encoded_image_path):
    # 读取载体图像和隐写图像
    img1 = cv2.imread(carrier_image_path, cv2.IMREAD_GRAYSCALE)
    img2 = cv2.imread(encoded_image_path, cv2.IMREAD_GRAYSCALE)
```

```

hist_ori = cv2.calcHist([img1], [0], None, [256], [0, 256]).flatten()
hist_stego = cv2.calcHist([img2], [0], None, [256], [0, 256]).flatten()
r_ori, p_ori = stgPrb(img1)
r_stego, p_stego = stgPrb(img2)
print(f"原始图像的 r 值:{r_ori}, P 值:{p_ori}")
print(f"隐写图像的 r 值:{r_stego}, P 值:{p_stego}")
return r_ori, p_ori, r_stego, p_stego

```

1.2.3 实现效果

嵌入率为 1 的情况下：

原始图像的 r 值：337.6170435673783，P 值：0.0

隐写图像的 r 值：47.642651706608525，P 值：0.9999999910240989

P 值接近于 1，表面图像中含有秘密信息。

且灰度直方图多处出现削峰，可以得出存在隐写的结论。

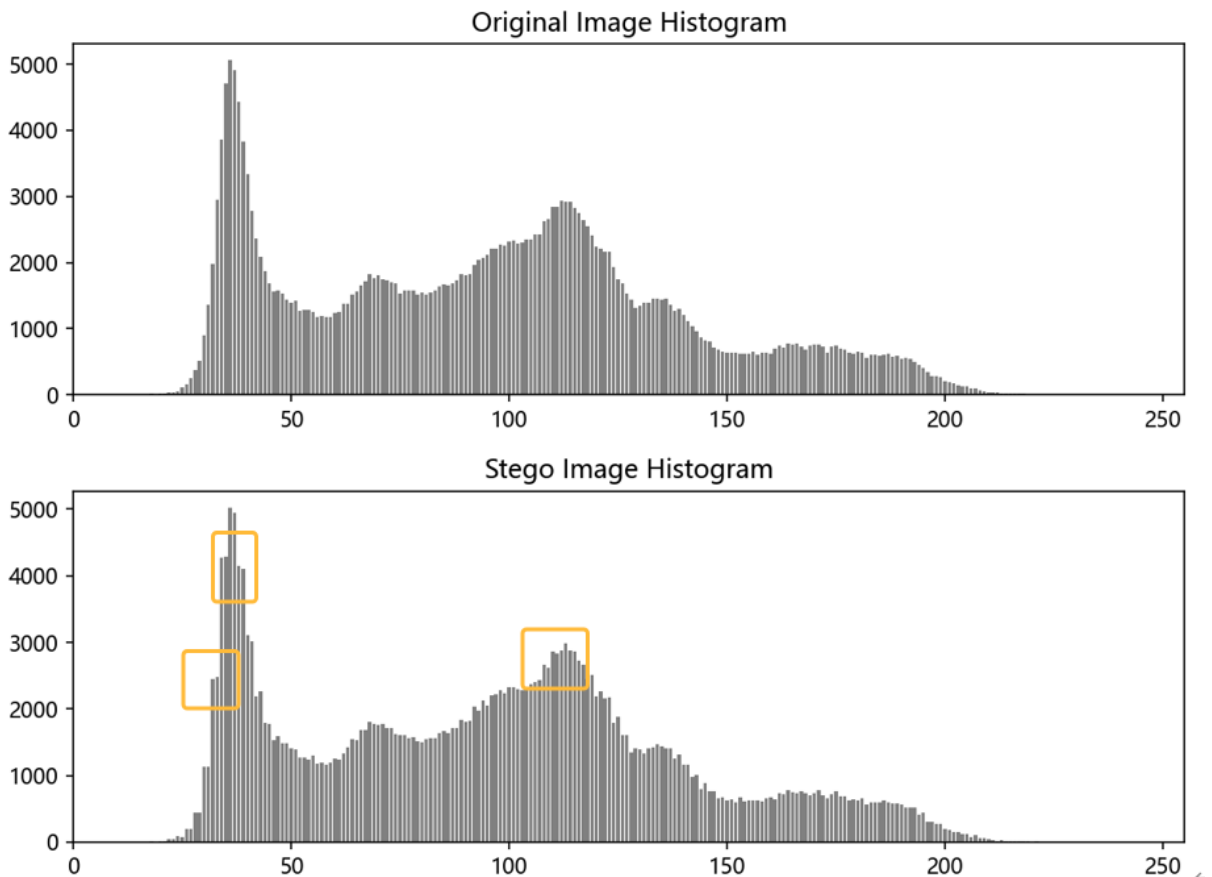


Figure 3lsb 顺序隐写嵌入率为 1

嵌入率为 0.5 的情况下：

原始图像的 r 值: 337.6170435673783, P 值: 0.0

隐写图像的 r 值: 200.46750481853942, P 值: $7.726632430538061e-07$

R 值接近, 且灰度图无明显差异, 不能判断出存在隐写。

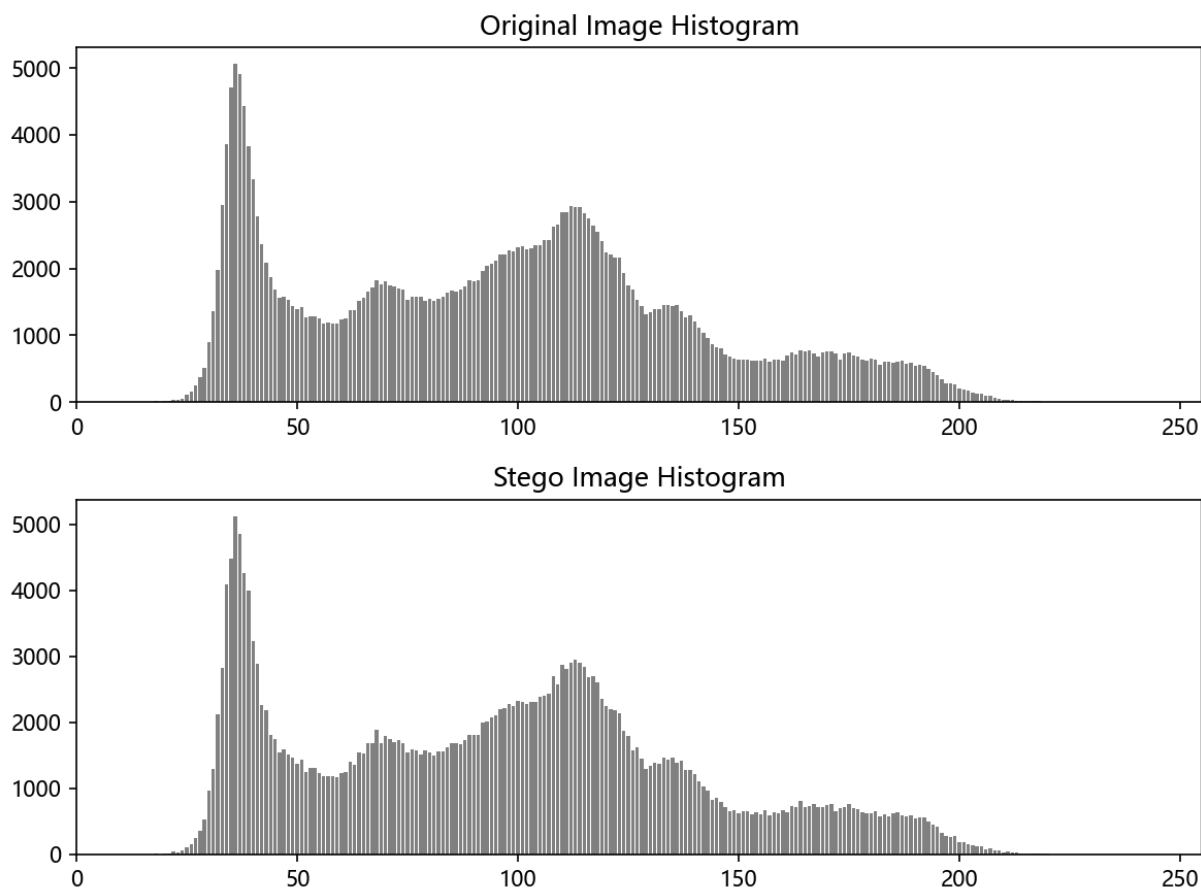


Figure 4lsb 顺序隐写嵌入率为 0.5

1.3 结论

LSB 隐写: 通过修改图像像素的最低有效位来嵌入秘密信息, 对视觉质量影响小。

大数据隐写时变化更明显, 因此检测隐写的可能性增大。在小数据隐写时, 变化较小, 不可以通过卡方分析检测出隐写痕迹。

卡方分析我认为仅仅适用于那种接近满嵌的图片, 他不适用于实际使用, 但可以作为一个评判标准。

任务二：完成对 BMP 位图格式图像文件的 LSB 和 MLSB 数据位的随机隐写并进行信息量估计法分析和 RS 分析。

2.1 LSB 随机隐写技术

2.1.1 概述

LSB 随机隐写是在 LSB 顺序隐写 的基础上增加了随机性，通过随机选择图像像素中的最低有效位（LSB）来隐藏数据，而不是按照顺序隐写。这增加了隐写的安全性，因为无法预测隐写数据的位置。

实现时，我们可以使用 `randi(256, 2, 8)`，生成一个 2x8 的矩阵，其中每个元素是从 1 到 256 的随机整数。然后根据这个矩阵来选择隐写数据的位置。

2.1.2 代码

```
# 打开灰度图像文件
im = Image.open(image_path).convert('L')
width, height = im.size
print(width, height)

# 从文件读取秘密信息并调整嵌入量
secret_data = read_secret_from_file(secret_file_path, embedding_rate,
width, height, 1)
secret_length = len(secret_data)
count = 0 # 用于跟踪已嵌入的比特数
pixels_list = [(w, h) for h in range(height) for w in range(width)]
random.shuffle(pixels_list) # 随机打乱像素坐标列表

while count < secret_length and pixels_list:
    w, h = pixels_list.pop() # 获取随机的像素坐标
    # 逐像素操作，获取灰度像素值
    pixel = im.getpixel((w, h))

    # 处理灰度像素
    if count < secret_length:
        pixel = (pixel & ~1) | int(secret_data[count]) # 清除最低有效位并嵌入比特
        count += 1
```

```
# 将修改后的像素写回  
im.putpixel((w, h), pixel)
```

2.1.3 实现效果

肉眼无区别。



Figure 6 左图为原图，右图为嵌入率为 0.6 的 lsb 随机隐写后的图片



Figure 5 左图为原图，右图为嵌入率为 0.2 的 lsb 随机隐写后的图片

进行卡方分析嵌入率为 1 时，灰度直方图明显区别

原始图像的 r 值：337.6170435673783， P 值：0.0

隐写图像的 r 值：63.11395980986767， P 值：0.9999601014422624

嵌入率为 0.2 时没有区别。

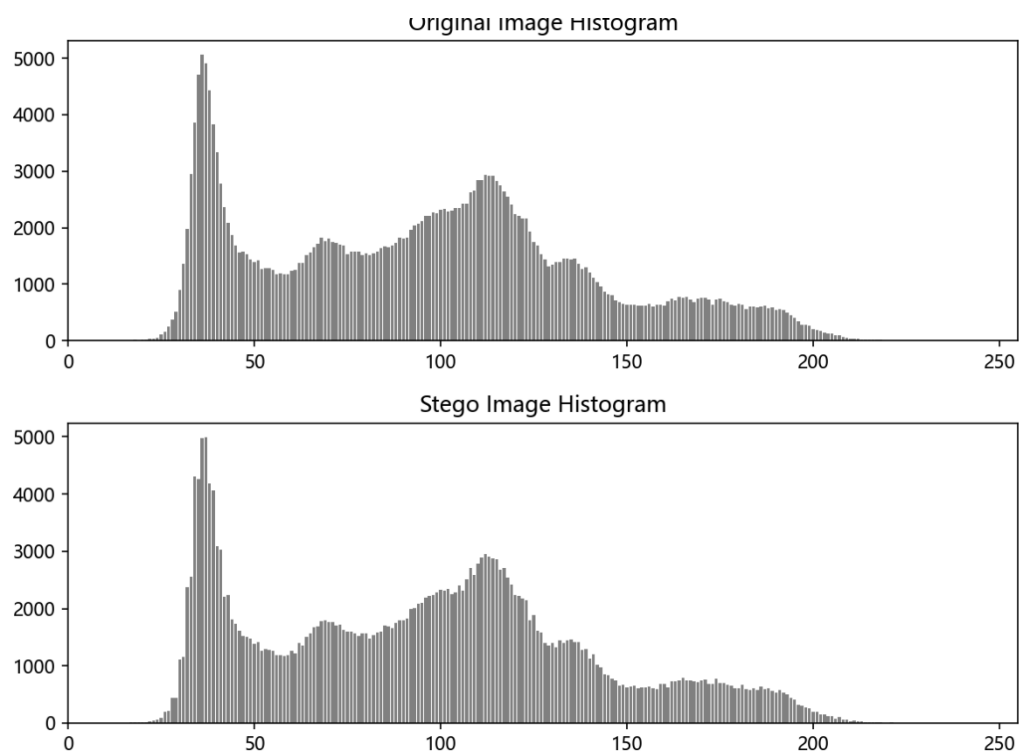


Figure 8 lsb 随机隐写嵌入率为 1

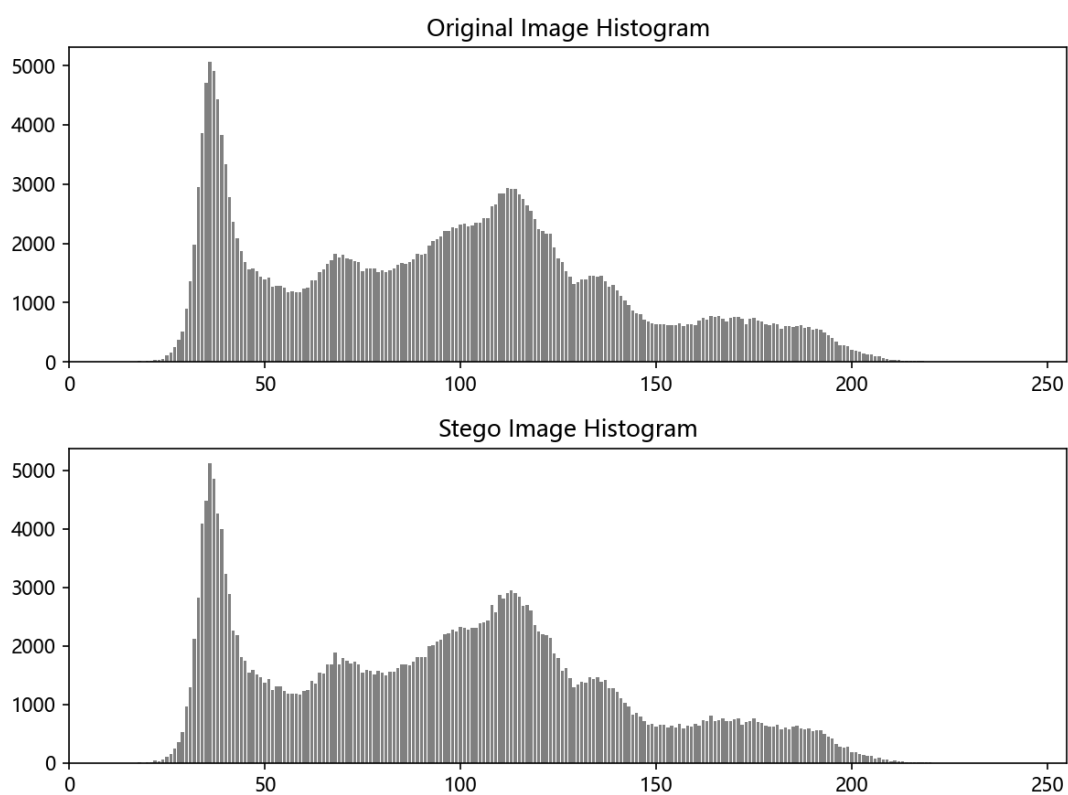


Figure 7 lsb 随机隐写嵌入率为 0.2

2.2 MLSB 随机隐写技术

2.2.1 MLSB 隐写原理

MLSB（多位最低有效位）隐写是基于 LSB 隐写的扩展，随机选择像素的一个位平面（0~7 位中的任意一位）嵌入秘密数据，而不仅限于最低有效位，通过在多个位上嵌入数据，可以提高隐写的容量，同时减少对图像质量的影响。

2.2.2 代码

实现 IMLSB 算法：我们首先获取所有像素的坐标，然后使用 `random.shuffle()` 来打乱这些坐标，确保隐写的信息分布在图像的各个位置，每次从打乱的像素列表中获取一个像素坐标（w, h），并读取其灰度值。

在嵌入数据时，我们会随机选择要嵌入的比特数，可能是 1 位（最低位）或 2 位（最低位和次低位）。这增加了隐写的复杂度，因为每个像素可以选择不同数量的比特来嵌入信息。。

```
# 获取所有像素并随机打乱顺序
pixels_list = [(w, h) for h in range(height) for w in range(width)]
random.shuffle(pixels_list) # 随机打乱像素坐标列表

while count < secret_length and pixels_list:
    w, h = pixels_list.pop() # 获取随机的像素坐标
    # 获取灰度像素值
    pixel = im.getpixel((w, h))

    # 处理灰度像素，每次嵌入 1 个比特到最低两位
    if count < secret_length:
        # 随机选择嵌入哪些比特位（最低一位，次低一位，或者两位都嵌入）
        embed_bits_choice = random.choice([1, 2]) # 随机选择 1 或 2 位

        # 获取要嵌入的比特
        embed_bits = int(secret_data[count:count + embed_bits_choice], 2) #
        获取相应数目的比特并转换为整数

        # 嵌入比特：如果选择 1 位，嵌入最低位；如果选择 2 位，分别嵌入最低位和次低位
        if embed_bits_choice == 1:
            # 嵌入最低一位
            pixel = (pixel & ~1) | embed_bits
        elif embed_bits_choice == 2:
            # 嵌入最低两位
```

```
pixel = (pixel & ~3) | embed_bits

count += embed_bits_choice # 每次嵌入 1 位或 2 位

# 将修改后的像素写回
im.putpixel((w, h), pixel)
```

2.2.3 实现效果

肉眼无区别。



Figure 10 左图为原图，右图为嵌入率为 0.6 的 mlsb 随机隐写后的图片



Figure 9 左图为原图，右图为嵌入率为 1 的 mlsb 随机隐写后的图片

进行卡方分析，满嵌时卡方分析明显，可以得出存在隐写痕迹的结论。

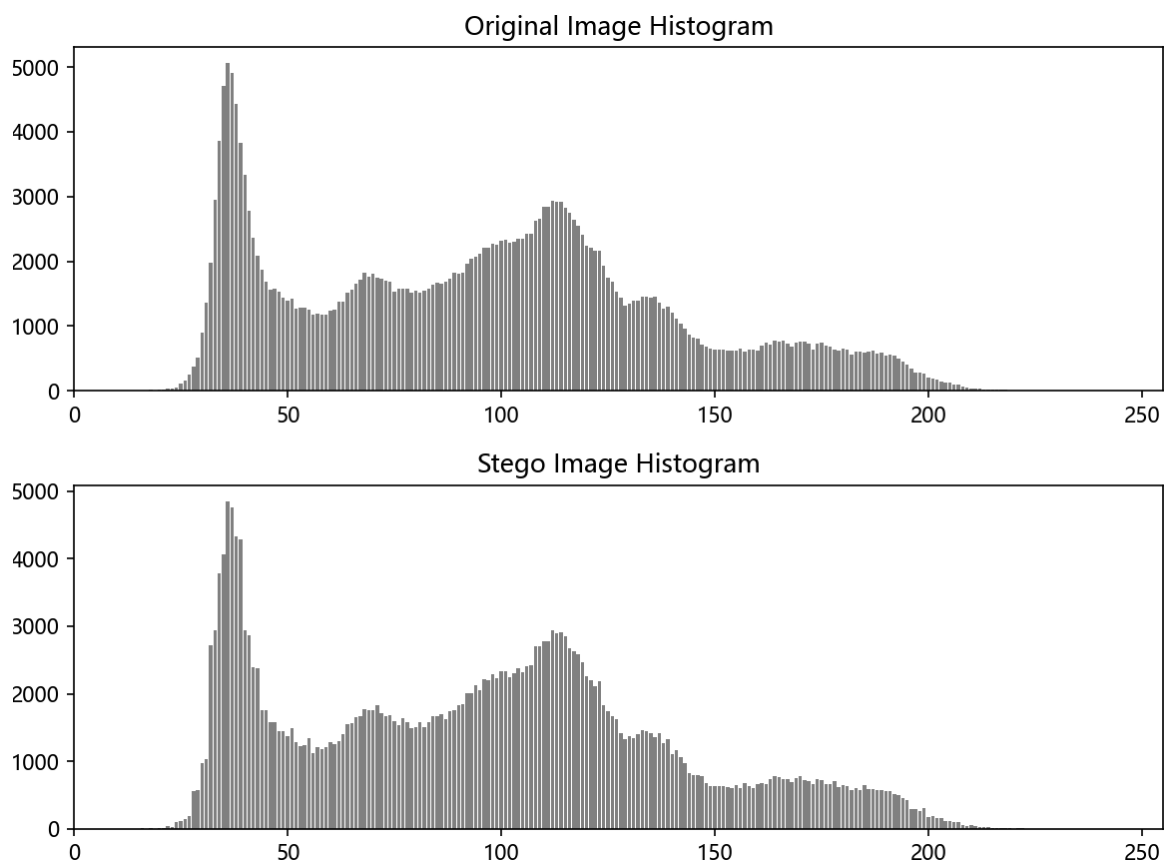


Figure 12 MLSB 随机隐写嵌入率为 1

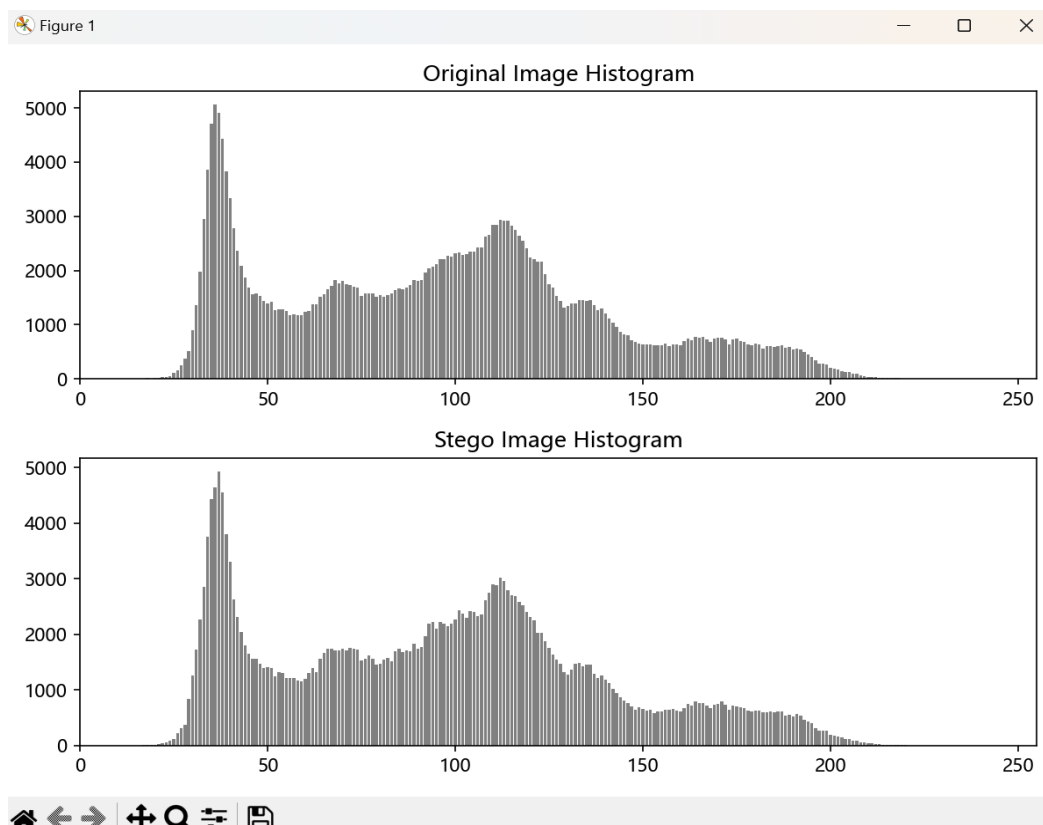


Figure 11 MLSB 随机隐写嵌入率为 0.2

嵌入率低时无法区分图片是否存在隐写_{f6}

2.3 信息量估计法分析

2.3.1 原理

信息量估计用于分析图像的直方图变化，特别是在隐写过程中，隐写信息嵌入图像后，图像的直方图会发生变化。通过对直方图的分析，可以了解嵌入信息后图像的变化程度，从而估计隐写的质量与可检测性。通过计算直方图的偶数和奇数位置的像素值的差异来估计信息的嵌入，如果图像经过隐写处理，其直方图与原图的直方图可能会有所不同。通过计算 $f1$ 和 $f2$ ，可以量化这些差异。然后通过二次隐写进行检测。

2.3.2 代码

```
def CalF(matrix):
    # 将矩阵展平并计算直方图
    n, _ = np.histogram(matrix.ravel(), bins=256, range=(0, 256))
    n = n.astype(int) # 确保直方图的值是整数

    # 计算 h(2i), h(2i+1), h(2i+2)
    h2i = n[0:255:2] # h(2i)
    h2ip1 = n[1:256:2] # h(2i+1)
    h2ip2 = n[2:256:2] # h(2i+2)

    # 计算 f1 和 f2
    f1 = 0
    f2 = 0

    for i in range(127):
        f1 += abs(h2ip1[i] - h2i[i])

    for i in range(126):
        f2 += abs(h2ip2[i] - h2ip1[i])

    return f1, f2

def LSB(cover, rate):
    # 将图像矩阵的类型转换成双精度型，便于后续的运算
    ste_cover = cover.astype(np.float64)

    # 读取 ste_cover 的行和列
    m, n = ste_cover.shape
```

```

# 根据 rate 计算秘密信息位的长度并生成秘密信息
msg_len = int(np.floor(m * n * rate))
msg = np.random.binomial(1, 0.5, msg_len) # 生成[0, 1]分布的随机数

p = 0 # p 为秘密信息的位计数器

# 按照产生的秘密信息依次将图片层的最后一位替换为秘密消息
for f2 in range(n):
    for f1 in range(m):
        if p >= msg_len:
            break
        ste_cover[f1, f2] = (ste_cover[f1, f2] - ste_cover[f1, f2] % 2) + msg[p]
        p += 1
    if p == msg_len:
        break

return ste_cover

```

2.3.3 实现效果

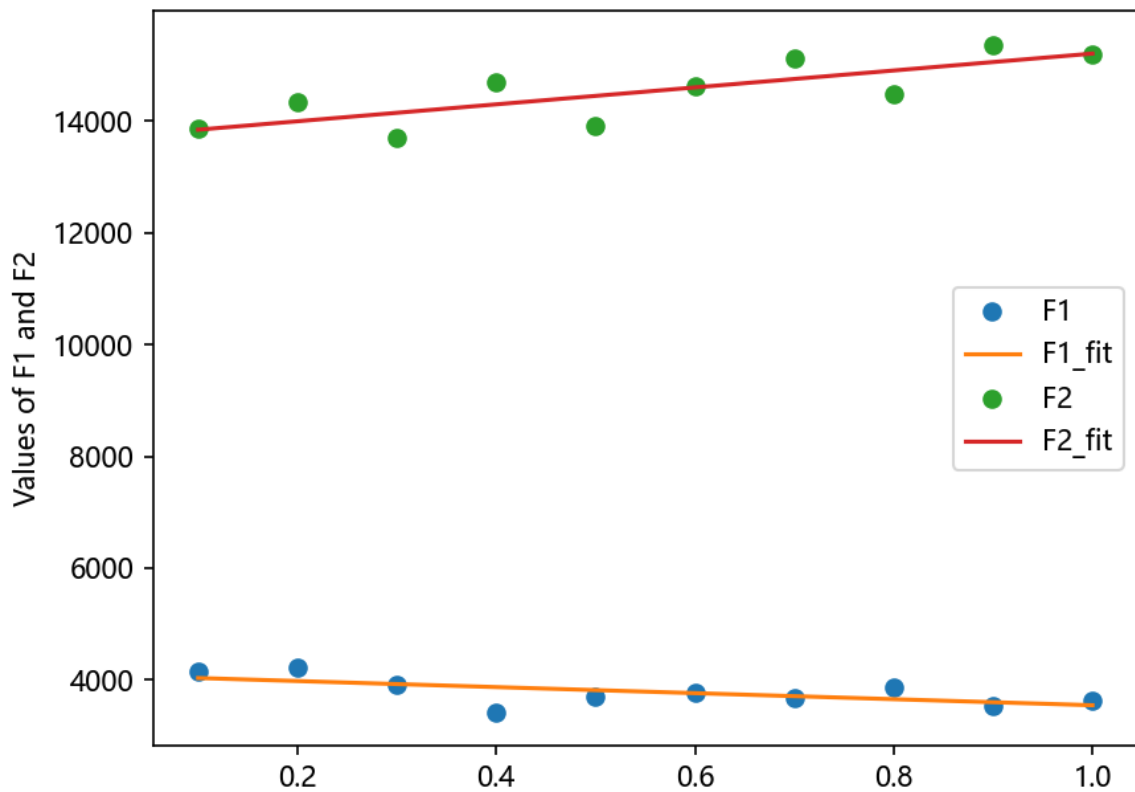


Figure 13 嵌入率为 0.9 的 mlsb 随机隐写

嵌入率为 0.9 的 m1sb 随机隐写通过分析嵌入率为 92%左右。

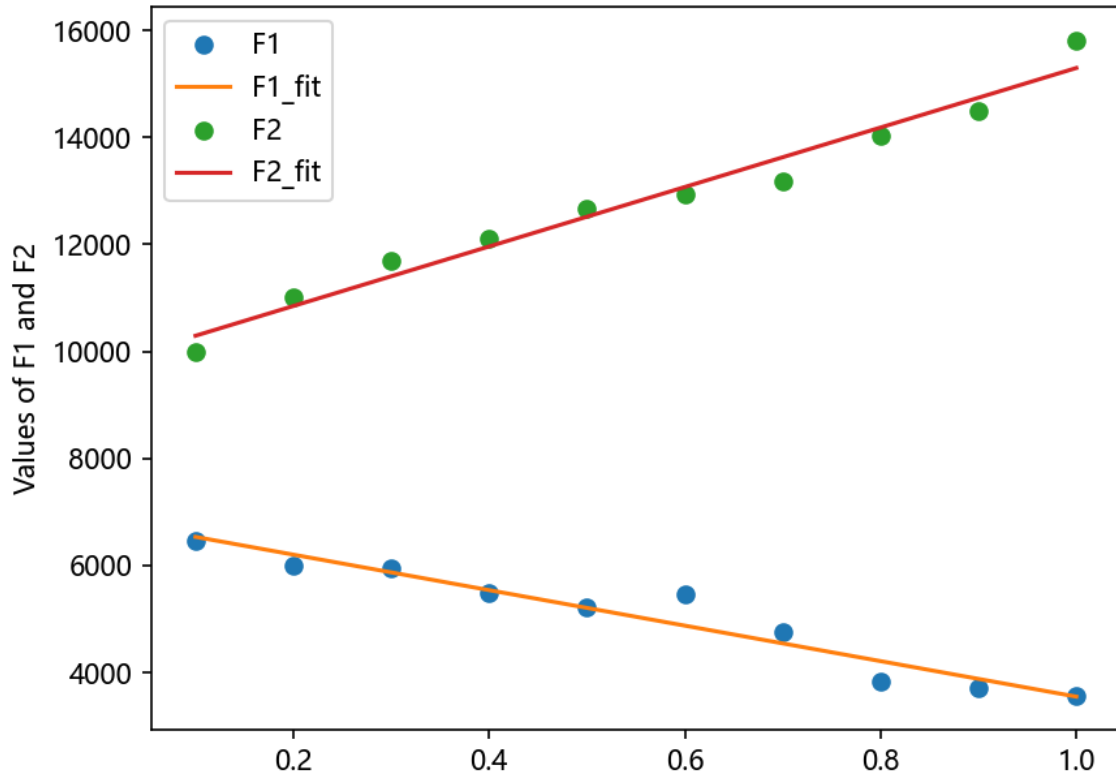


Figure 14 lsb 随机隐写 0.2

嵌入率为 0.2 的 m1sb 随机隐写嵌入率为 35%左右

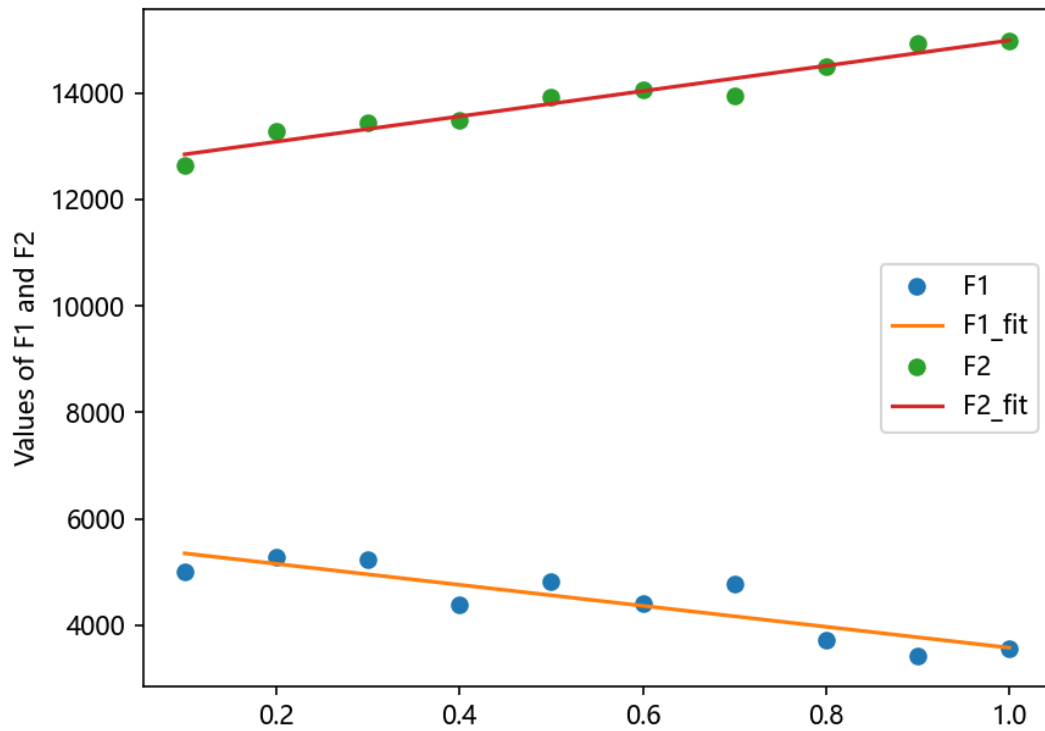


Figure 15 lsb 随机隐写嵌入率为 0.9

嵌入率为 0.9 的 1sb 随机隐写通过分析嵌入率为 92%左右

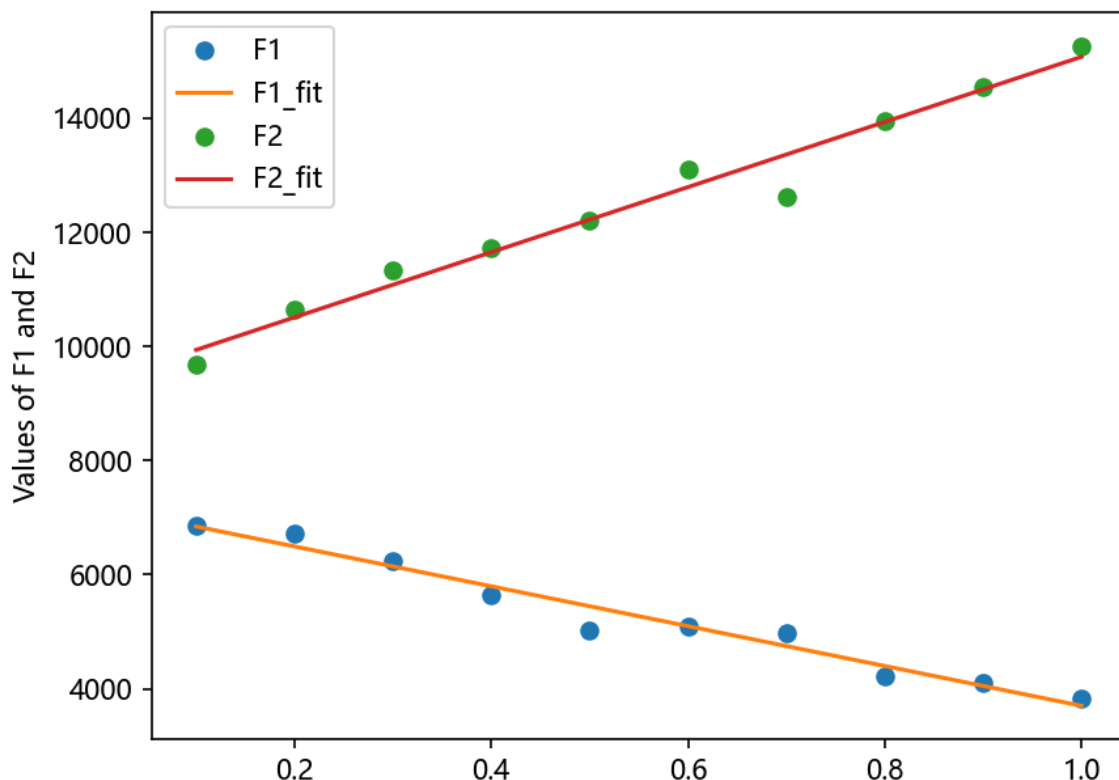


Figure 16 1sb 随机隐写嵌入率为 0.2

嵌入率为 0.2 的 1sb 随机隐写通过分析嵌入量为 17%左右

f1 和 f2: 通过这两个值的差异, 判断图像是否含有秘密信息。如果 f2 值大于 f1 并且差距显著, 那么可能意味着图像的最低有效位被修改过, 秘密信息可能已被嵌入。

若不存在隐写, 则 f1 和 f2 初始值接近, 没有显著差距。

2.4 RS 分析

2.4.1 原理

隐写技术通过修改图像的某些像素或频率成分来嵌入数据, 这些操作往往对图像的高频部分 (如边缘信息、纹理细节等) 有较小的影响, 但可能会影响到低频部分, 尤其是隐写操作如 LSB (最低有效位) 修改、DCT (离散余弦变换) 嵌入等。隐写信息的嵌入通常会引入以下变化:

低频信息: 隐写信息在频域的低频部分可能会引起数据的改变, 尤其是隐写信息的嵌入位置选择和压缩比的不同。

高频噪声: 隐写操作可能会产生某些高频噪声, 改变图像的原始频谱特征。

频率的分布变化：正常图像在频域中具有特定的频率分布，隐写信息的嵌入可能会打破这种分布模式。

RS 分析的核心在于通过频域统计来检测隐写信息。

2.4.2 代码

```
# 对像素块进行 z 字排序
def Z(tmp):
    size = 8
    a = [0] * 64
    n = 0
    i = 0
    j = 0
    status = 0 # 标记当前状态 0: 右上运动 1: 左下运动
    while (n < size * size):
        if ((i == 0 and j % 2 != 0) or (j == 1 and i == 0)) and j != size - 1:
            # 当前位于像素矩阵的最左边的奇数位且不是边界，向下移动，下一步向右上运动
            a[n] = tmp[i][j]
            j = j + 1
            n = n + 1
            status = 0
        if ((i == 0 and j % 2 != 0) or (j == 1 and i == 0)) and j == size - 1:
            # 当前位于像素矩阵的最左边的奇数位且是边界，向右移动，下一步向右上运动
            a[n] = tmp[i][j]
            i = i + 1
            n = n + 1
            status = 0
        elif ((j == 0 and i % 2 == 0 and i > 1) or (i == 0 and j == 0)): # 当前位于像素矩阵的最上面的偶数位，向右移动，下一步向左下运动
            a[n] = tmp[i][j]
            i = i + 1
            n = n + 1
            status = 1
        elif ((i == 0 and j % 2 == 0 and j > 1) or (j == 0 and i == 0)): # 当前位于像素矩阵的最左边的偶数位，向右上移动
            a[n] = tmp[i][j]
            j = j - 1
            i = i + 1
            n = n + 1
            status = 0
        elif ((j == 0 and i % 2 != 0) or (i == 1 and j == 0)): # 当前位于像素矩阵的最上面的奇数位，向左下移动
```

```

        a[n] = tmp[i][j]
        i = i - 1
        j = j + 1
        n = n + 1
        status = 1

    elif ((i == size - 1 and j % 2 != 0)): # 当前位于像素矩阵的最右边的奇数
位, 向下移动, 下一步向左下运动
        a[n] = tmp[i][j]
        j = j + 1
        n = n + 1
        status = 1

    elif ((j == size - 1 and i % 2 == 0 and i > 1)): # 当前位于像素矩阵的最
下面的偶数位, 向右移动, 下一步向右上移动
        a[n] = tmp[i][j]
        i = i + 1
        n = n + 1
        status = 0

    elif ((i == size - 1 and j % 2 == 0 and j > 1)): # 当前位于像素矩阵的最
右边的偶数位, 向左下移动
        a[n] = tmp[i][j]
        i = i - 1
        j = j + 1
        n = n + 1
        status = 1

    elif ((j == size - 1 and i % 2 != 0)): # 当前位于像素矩阵的最下面的奇数
位, 向右上移动
        a[n] = tmp[i][j]
        i = i + 1
        j = j - 1
        n = n + 1
        status = 0

    else: # 不是边界条件时, 使用状态值判断移动方向
        if (status == 0): # 右上运动
            a[n] = tmp[i][j]
            i = i + 1
            j = j - 1
            n = n + 1
            status = 0

        elif (status == 1): # 左下运动
            a[n] = tmp[i][j]
            i = i - 1
            j = j + 1
            n = n + 1

```

```
        status = 1
    return a

# 计算像素相关性
def Calculate(a):
    res = 0
    for i in range(63):
        if (a[i + 1] > a[i]):
            res = res + a[i + 1] - a[i]
        else:
            res = res + a[i] - a[i + 1]
    return res

# 0 翻转
def F0(val):
    return val

# 正翻转
def F1(val):
    if (val % 2 == 0 and val != 1): # 偶数加一
        val = val + 1
    elif (val % 2 == 1 or val == 1): # 奇数减一
        val = val - 1
    return val

# 负翻转
def F_1(val):
    if (val % 2 == 0 and val != 1): # 偶数减一
        val = val - 1
    elif (val % 2 == 1 or val == 1): # 奇数加一
        val = val + 1
    return val

# 生成随机数组
def Random(typ):
    ran = [0] * 64
    if (typ == 1):
        for i in range(64):
```



```

        ran[i] = random.randint(0, 1)
    elif (typ == -1):
        for i in range(64):
            ran[i] = random.randint(-1, 0)
    return ran

# RS 隐写分析
def RS(tmp):
    #
    ran = Random(1)
    rev = [[0 for co in range(8)] for ro in range(8)] # 进行反转后的二维数组
    rm = 0
    sm = 0

    r1 = Z(tmp)
    res1 = Calculate(r1) # 反转之前的像素相关性

    k = 0
    for i in range(8):
        for j in range(8):
            if (ran[k] == 0): # F0 翻转
                rev[i][j] = F0(tmp[i][j])
            elif (ran[k] == 1): # F1 翻转
                rev[i][j] = F1(tmp[i][j])
            k = k + 1

    r2 = Z(rev) # 将图像块进行 z 字形排序
    res2 = Calculate(r2) # 翻转之后的像素相关性

    if (res1 > res2):
        sm = sm + 1
    elif (res1 < res2):
        rm = rm + 1

    # =====
    k = 0
    r_m = 0
    s_m = 0
    ran = Random(-1)
    rev = [[0 for co in range(8)] for ro in range(8)] # 进行反转后的二维数组
    for i in range(8):
        for j in range(8):
            if (ran[k] == 0):

```

```

        rev[i][j] = F0(tmp[i][j])
    elif (ran[k] == -1):
        rev[i][j] = F_1(tmp[i][j])
    k = k + 1

r3 = Z(rev) # 将图像块进行 z 字形排序
res3 = Calculate(r3) # 翻转之后的像素相关性

if (res1 > res3):
    s_m = s_m + 1
elif (res1 < res3):
    r_m = r_m + 1

res = [rm, sm, r_m, s_m]
return res

def rsd(path, iterations=20):
    # 读取图像
    origin_rgb = mpimg.imread(path)
    origin = np.array(origin_rgb[:, :, 0], dtype=np.float64) # 确保使用
float64, 避免溢出

    row = origin.shape[0] // 8 # 行
    col = origin.shape[1] // 8 # 列

    # 初始化相关性结果
    result = [0.0] * 4 # 确保 result 是浮点数类型
    rm = [0.0] * 11
    sm = [0.0] * 11
    r_m = [0.0] * 11
    s_m = [0.0] * 11
    tmp = np.zeros((8, 8), dtype=np.float64) # 用于存储 8x8 的图像块

    # 初始化总的隐写率累加器
    total_estimated_rate = 0.0

    for _ in range(iterations):
        # 每次循环重新初始化 result 和临时数组
        result = [0.0] * 4
        # 对每个 8x8 块进行 RS 隐写分析
        for i in range(row):
            for j in range(col):

```

```

x = 0
for k in range(i * 8, (i + 1) * 8):
    y = 0
    for h in range(j * 8, (j + 1) * 8):
        tmp[x][y] = origin[k][h]
        y += 1
    x += 1

# 执行 RS 隐写分析
res = RS(tmp) # 假设 RS() 函数返回一个包含 4 个结果的列表
for n in range(4):
    result[n] += res[n]

# 计算 d0, d1, d2, d3
d0 = result[0] - result[1]
d1 = rm[0] - sm[0]
d2 = result[2] - result[3]
d3 = r_m[0] - s_m[0]

# 计算二次方程的系数
args = [2 * (d1 + d0), d2 - d3 - d1 - 3 * d0, d0 - d2]

# 使用 numpy 的 roots 方法求解二次方程
a, b = np.roots(args)

# 估计隐写率
if a > b:
    estimated_rate = b / (b - 0.5)
else:
    estimated_rate = a / (a - 0.5)

# 累加估计的隐写率
total_estimated_rate += estimated_rate
average_estimated_rate = total_estimated_rate / iterations

# 输出平均隐写率
print(f"估计隐写率的平均值为: {average_estimated_rate}")

```

2.4.3 实现效果

嵌入率为 1 的 m1sb 随机隐写 RS 分析

估计隐写率的平均值为：0.8436411833820868

嵌入率为 0.1 的 mlsb 随机隐写 RS 分析

估计隐写率的平均值为：0.15467988110391753

嵌入率为 1 的 lsb 顺序隐写 RS 分析

估计隐写率的平均值为：0.9136304659069694

嵌入率为 0.1 的 lsb 顺序隐写 RS 分析

估计隐写率的平均值为：0.16706409715314485

嵌入率为 0.4 的 lsb 随机隐写 RS 分析

估计隐写率的平均值为：0.3977493900650443

嵌入率为 0.2 的 lsb 随机隐写 RS 分析

估计隐写率的平均值为：0.25067256597634213

2.5 实验结果与讨论

LSB 随机隐写是在 LSB 顺序隐写的基础上增加随机性，提高安全性。

MLSB 随机隐写是扩展 LSB 隐写算法，它修改多个位平面，提高隐写容量，但如果选择的位面越多越可能影响图像质量。

信息量估计法分析：通过比较隐写前后的熵值和 PSNR，评估隐写对图像信息量和质量的影响。结果显示，隐写对信息量和质量的影响很小。

研究 LSB 隐写法随着隐写率的变化，RS 分析结果的变化趋势。

对图像的每个 8x8 小块，进行 0-1 嵌入率的逐增

```
rate = 0

while rate <= 1.0:
    fig = LSB(fig, rate) # 对图像进行 LSB 隐写
```

```

result = [0] * 4
for i in range(64):
    for j in range(64):
        x = 0
        for k in range(i * 8, (i + 1) * 8):
            y = 0
            for h in range(j * 8, (j + 1) * 8):
                tmp[x][y] = fig[k][h]
                y += 1
            x += 1
        res = RS(tmp) # 使用 RS 隐写分析
        for n in range(4):
            result[n] += res[n]
    # 打印当前隐写率的结果
    print(f'rate={rate} after LSB:')
    print('rm=', result[0], ' sm=', result[1], ' r_m=', result[2], '
s_m=', result[3])

d0 = result[0] - result[1]
d1 = result[2] - result[3]
d2 = rm[0] - sm[0]
d3 = r_m[0] - s_m[0]
epsilon = 1e-6 # 设置一个容忍度, 防止 b 非常接近 0.5

# 计算二次方程的参数
args = [2 * (d1 + d0), d2 - d3 - d1 - 3 * d0, d0 - d2]
b = (-args[1] - np.sqrt(args[1] ** 2 - 4 * args[0] * args[2])) / (2 *
args[0])

# 检查 b 是否接近 0.5, 避免 b=0.5 的情况
if abs(b - 0.5) < epsilon:
    print("Warning: b is very close to 0.5, skipping estimation.")
    estimated_rate = 0 # 或者设置为某个默认值, 例如 0, 表示无法估算
else:
    estimated_rate = b / (b - 0.5)

print(f"Estimated rate = {estimated_rate}")
if rate == 0:
    print(b)
    estimated_rate = 0

# 累加隐写率
rsrate_total += estimated_rate

```

```

rm[n] = result[0]
sm[n] = result[1]
r_m[n] = result[2]
s_m[n] = result[3]

n += 1
rate += 0.1

```

可以得到rm、sm、r_m和s_m随嵌入率变化的数值并绘制成以下图片。

rm: 蓝色曲线，表示正向翻转的相关性得分。随着嵌入率的增加，相关性得分呈下降趋势，这可能表明隐写信息的增加导致图像规律性的减少。

sm: 橙色曲线，表示负向翻转的相关性得分。这条曲线随着嵌入率的增加而上升，可能意味着负向翻转在高嵌入率下更能保持图像的规律性。

r_m: 绿色曲线，表示正向翻转且非负向的相关性得分。这条曲线随着嵌入率的增加而上升，表明在高嵌入率下，正向翻转的非负向特性变得更加明显。

s_m: 红色曲线，表示负向翻转且非正向的相关性得分。这条曲线随着嵌入率的增加而下降，可能表明负向翻转的非正向特性在高嵌入率下被削弱。

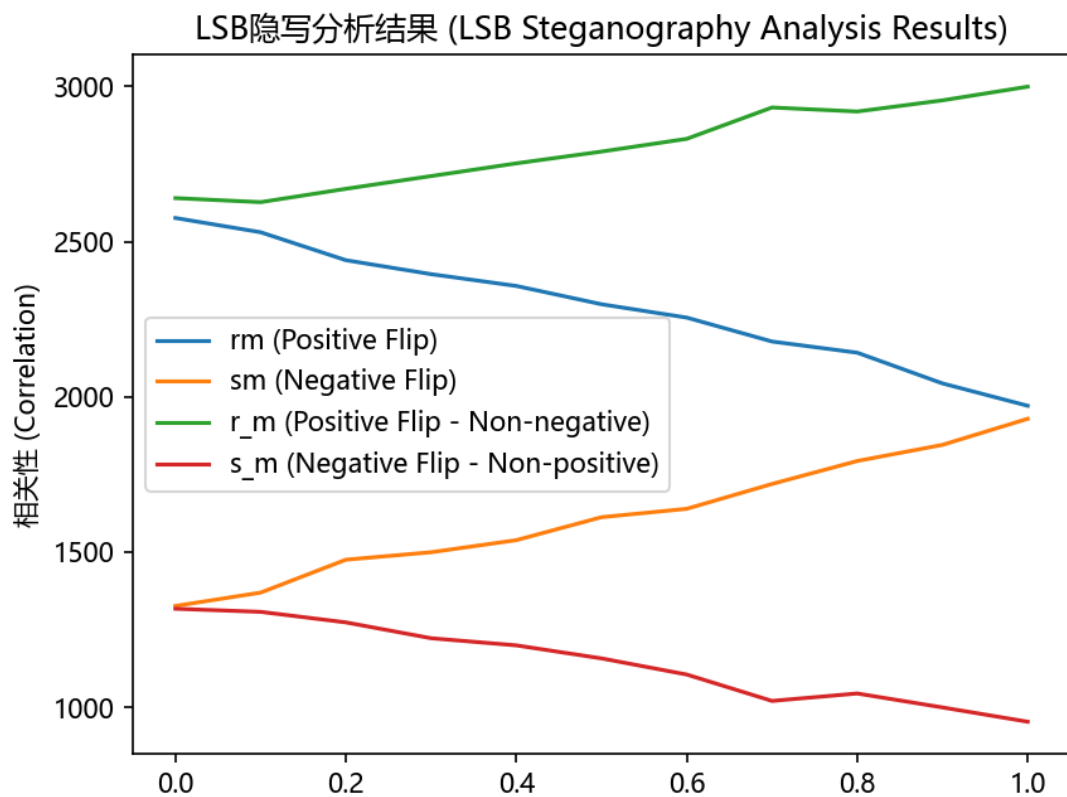


Figure 17 原图

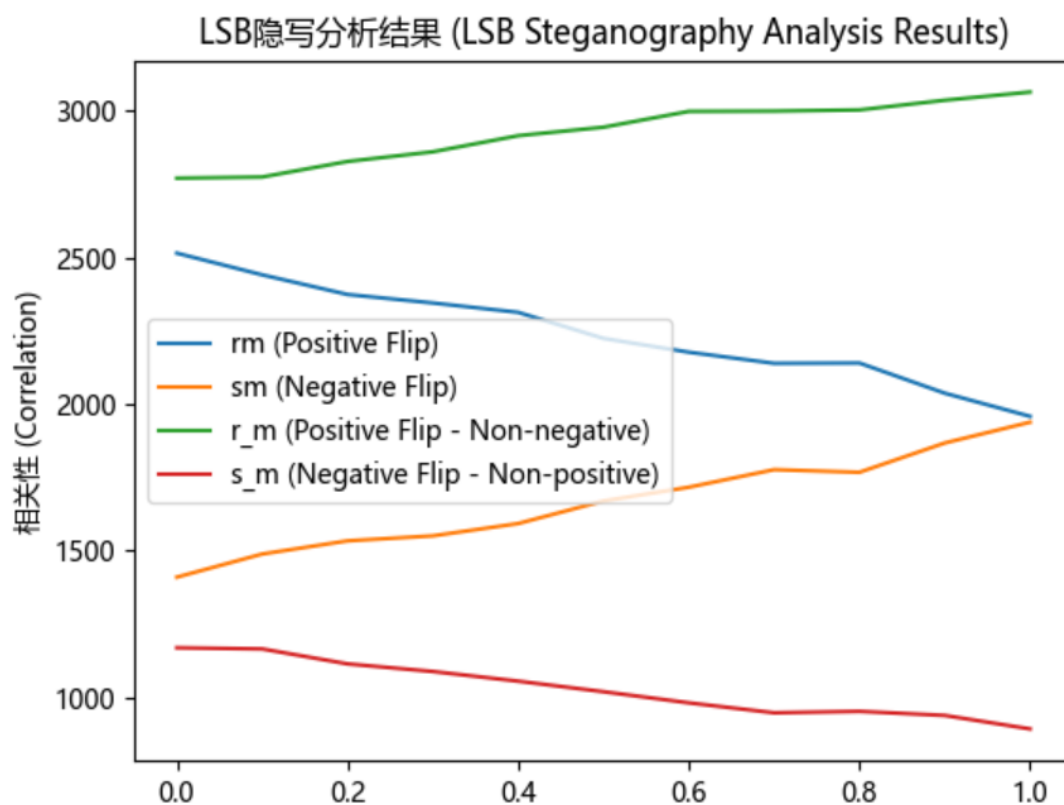


Figure 18 嵌入率为 0.2

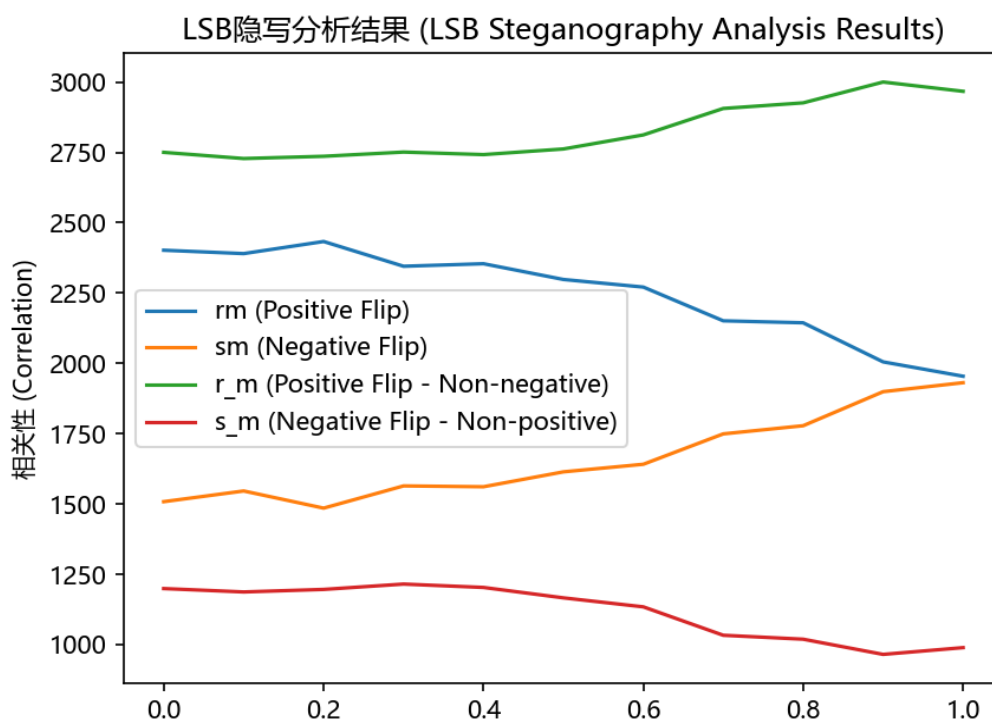


Figure 19 嵌入率为 0.3

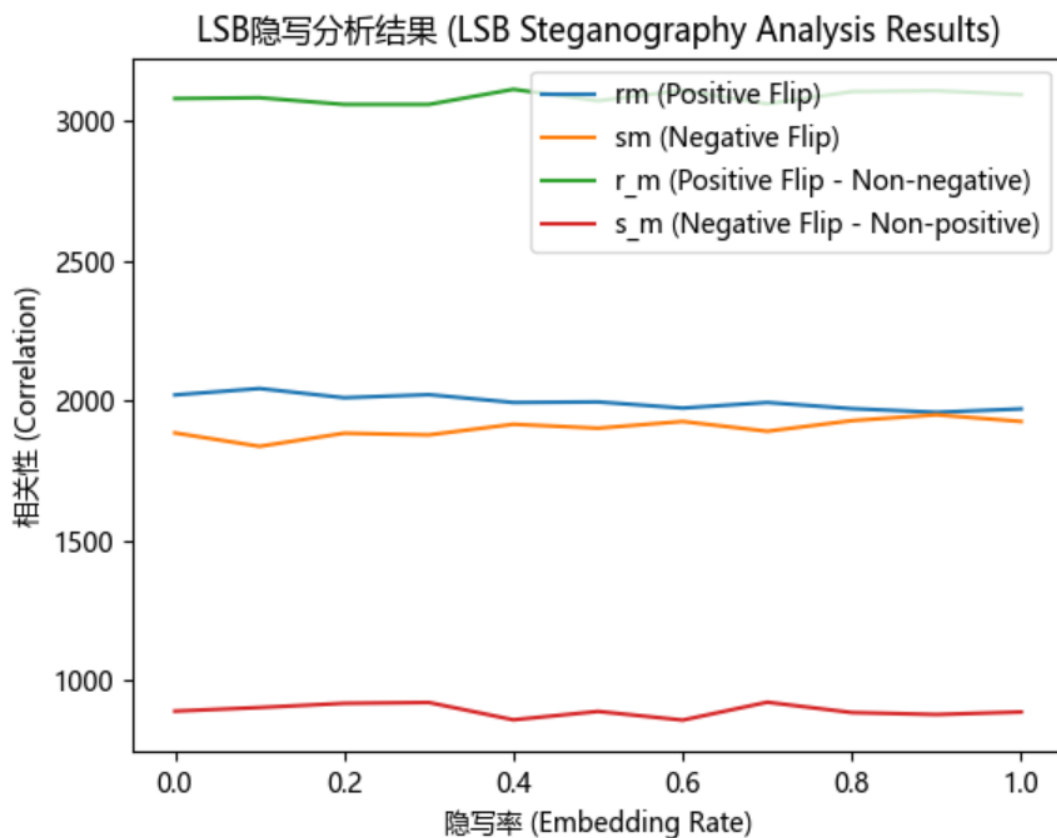


Figure 20 嵌入率为 0.9

正向翻转的相关性得分普遍下降，而负向翻转的相关性得分则上升。这可能意味着在 LSB 隐写过程中，图像的某些规律性被改变或破坏，而 RS 分析能够捕捉到这些变化。

总结与比较：

分析方法	适用场景	优点	缺点
卡方分析	- 适用于检测隐写数据对图像像素分布的影响 - 适用于大数据隐写检测	- 计算简单 - 可检测隐写对像素分布的影响	- 对隐写数据量少或数据隐藏方式隐蔽的效果差
RS 分析	- 适用于基于频域的隐写检测 - 检测隐写对图像频谱的影响	- 适用于检测细微的隐写信息变化 - 可以检测频域中低频与高频的变化	- 对压缩图像效果差 - 隐写信息在频域的嵌入方式较难检测
信息量	- 适用于评估数据的嵌入量	- 精确估算隐写数据量	- 对压缩图像效果差

分析方 法	适用场景	优点	缺点
估计法	- 适用于检测隐写容量	- 评估图像隐写后的信 息完整性	- 隐写数据较少时可能不易检 测

任务三：完成对 BMP 位图格式图像文件的抗分析的 LSB 数据位的随机隐写和分析

3.1 抗 RS 分析

3.1.1 代码

在 LSB 随机隐写的基础上，我们进行修改。

处理像素时，不是简单的抹掉最低位进行覆盖秘密信息，而是对于每个通道，代码检查当前像素值的最低有效位：

- 如果 LSB 为 0，则直接将信息的 LSB 写入，不改变像素值。
- 如果 LSB 为 1，则根据周围像素的平均值决定是增加还是减少当前像素值。通过计算以当前像素为中心的 3x3 区域内相应通道值的总和，然后与当前像素值的 9 倍进行比较来实现的。如果总和小于或等于当前像素值的 9 倍，则减少像素值；否则，增加像素值。

```
• if self.mod(a, 2) == 0:
    count += 1
    if count == keylen:
        im.putpixel((w, h), (a, b, c))
        break
    else:
        if a == 0:
            a = a + 1
            count += 1
            if count == keylen:
                im.putpixel((w, h), (a, b, c))
                break
        elif a == 255:
            a = a - 1
            count += 1
            if count == keylen:
                im.putpixel((w, h), (a, b, c))
                break
        else:
            T = 0
            for u in range(w - 1, w + 2):
                for v in range(h - 1, h + 2):
                    p = im.getpixel((u, v))
                    T = T + p[0]
            T = T - 9 * a
            if T <= 0:
                a = a - 1
                count += 1
```

```
        if count == keylen:
            im.putpixel((w, h), (a, b, c))
            break
    else:
        a = a + 1
        count += 1
        if count == keylen:
            im.putpixel((w, h), (a, b, c))
            break
if self.mod(b, 2) == 0:
    count += 1
    if count == keylen:
        im.putpixel((w, h), (a, b, c))
        break
    else:
        if b == 0:
            b = b + 1
            count += 1
            if count == keylen:
                im.putpixel((w, h), (a, b, c))
                break
        elif b == 255:
            b = b - 1
            count += 1
            if count == keylen:
                im.putpixel((w, h), (a, b, c))
                break
        else:
            T = 0
            for u in range(w - 1, w + 2):
                for v in range(h - 1, h + 2):
                    p = im.getpixel((u, v))
                    T = T + p[1]
            T = T - 9 * b
            if T <= 0:
                b = b - 1
                count += 1
                if count == keylen:
                    im.putpixel((w, h), (a, b, c))
                    break
            else:
                b = b + 1
                count += 1
```

```

        if count == keylen:
            im.putpixel((w, h), (a, b, c))
            break
    if self.mod(c, 2) == 0:
        count += 1
        if count == keylen:
            im.putpixel((w, h), (a, b, c))
            break
    else:
        if c == 0:
            c = c + 1
            count += 1
            if count == keylen:
                im.putpixel((w, h), (a, b, c))
                break
        elif c == 255:
            c = c - 1
            count += 1
            if count == keylen:
                im.putpixel((w, h), (a, b, c))
                break
        else:
            T = 0
            for u in range(w - 1, w + 2):
                for v in range(h - 1, h + 2):
                    p = im.getpixel((u, v))
                    T = T + p[2]
            T = T - 9 * c
            if T <= 0:
                c = c - 1
                count += 1
                if count == keylen:
                    im.putpixel((w, h), (a, b, c))
                    break
            else:
                c = c + 1
                count += 1
                if count == keylen:
                    im.putpixel((w, h), (a, b, c))
                    break
    if count % 3 == 0:
        im.putpixel((w, h), (a, b, c))

```

通过随机选择像素和根据周围像素值调整当前像素值来减少 LSB 替换的可检测性。

这种方法试图在不引入明显相关性的同时嵌入信息，尽量减少像素值的变化，从而减少图像的视觉失真，并降低被统计分析方法检测到的风险。

3.1.2 分析

- 肉眼观察两幅图，没有区别。



Figure 21 左图为原图，右图为嵌入率为 1 的抗 rs 隐写后的图片

- 卡方分析

原始图像的 χ^2 值：219.96650308072032，P 值：0

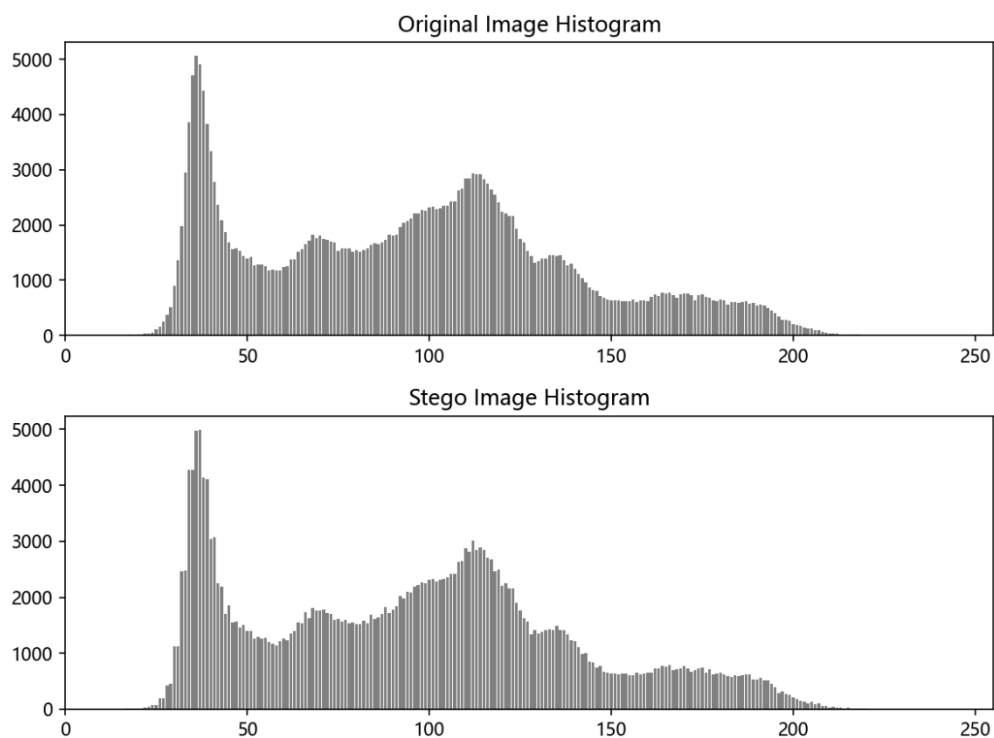


Figure 22 抗 rs 分析嵌入率为 1

隐写图像的 r 值: 221.00564539310892, P 值: 0.9999951173304977

P 值接近于 1, 表明存在隐写。

隐写后的灰度直方图出现削峰, 表面存在隐写。

即该抗 RS 分析的代码仍能被卡方分析检测到。

● 信息量估计分析

PSNR 比较了原始图像和处理后图像之间的差异。PSNR 值越高, 表示两个图像之间的差异越小, 图像质量越好。51.95 dB 的 PSNR 值表明隐写过程对图像质量的影响很小。

隐写前后熵值变化很小 (从 7.4455 降到 7.4395), 这表明隐写过程对图像整体信息量的影响不大。

隐写图像的峰值信噪比 (PSNR) 为: 51.95 dB

原始图像的香农熵为: 7.4455

隐写图像的香农熵为: 7.4395

进行信息量估计计算 $F1$ 和 $F2$ 初始值

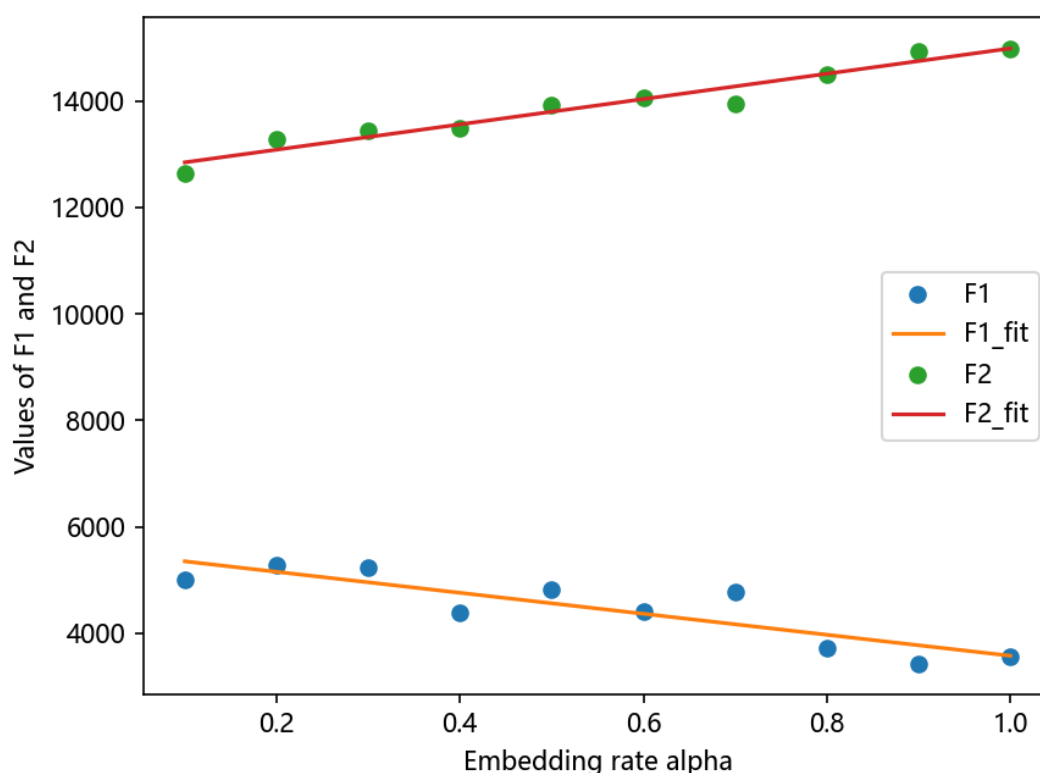


Figure 23 抗 rs 分析嵌入率为 1

即该抗 RS 分析的代码能被信息量估计分析法检测到。

● RS 分析

抗 rs 分析的 lsb 随机算法

估计隐写率的平均值为: 0.006604471757327165

Lsb 随机算法

估计隐写率的平均值为: 0.9136304659069694

Rs 分析图与原图进行 rs 分析类似, 判断不出是否进行了隐写。

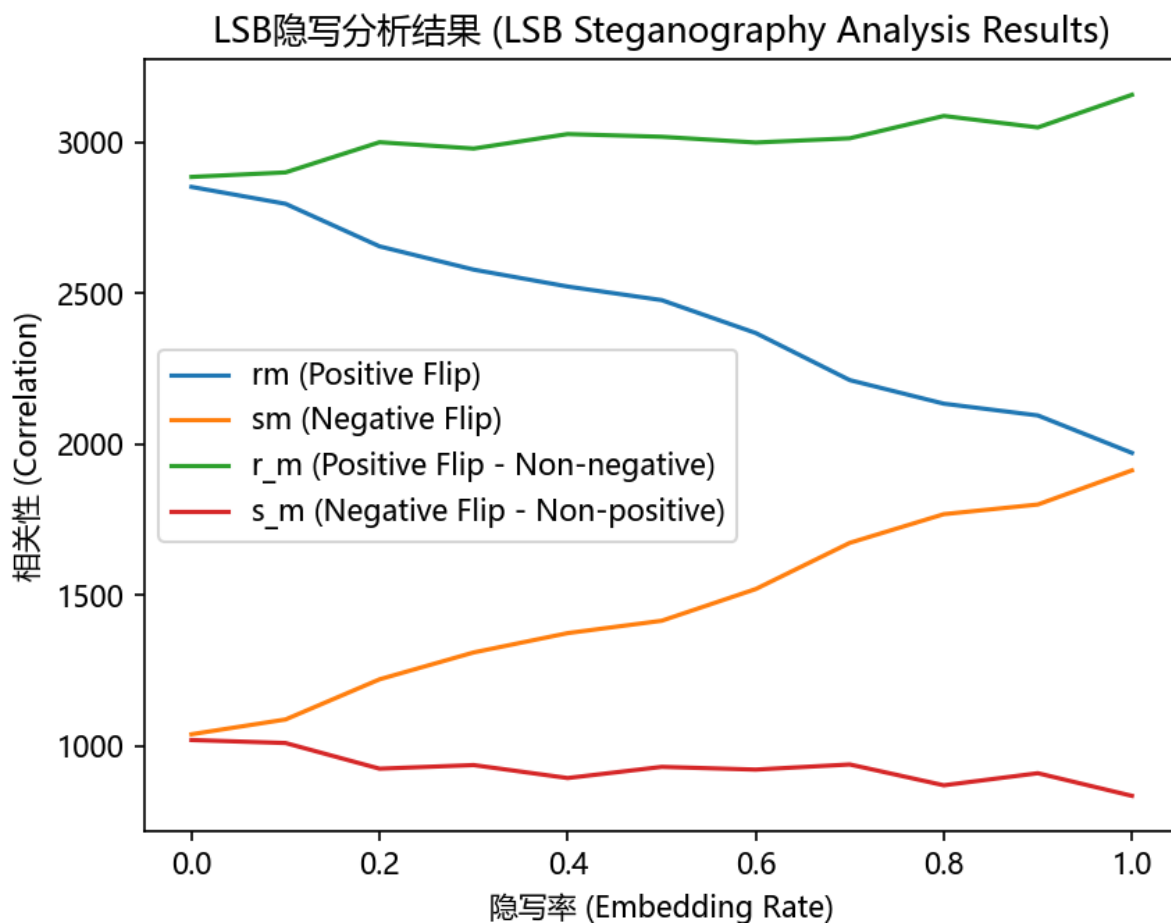


Figure 24 抗 rs 分析的 rs 分析

原 lsb 隐写算法经过 RS 分析检测结果为 0.9136304, 而该抗 rs 分析算法经过 RS 分析检测结果为-0.0032155, 表明隐写信息被成功隐藏或移除, 使得图像看起来更加随机。

3.1 抗卡方分析和信息量估计分析代码

f_z 和 f_{z+1} 代表不同像素值的频率。通常, LSB 隐写会直接影响最低有效位, 因此, 频繁修改最低有效位的像素会引起图像的统计偏差, 容易被检测。当不含秘密信息的 $2i+1$ 像素值过多时, 通过将其转化为 $2i$ 来降低隐写痕迹。这种补偿方法可以在嵌

入数据时保持图像的统计特征相对一致，从而提升隐写的隐蔽性。

过程：

- 遍历每个像素，通过其最低有效位（LSB）嵌入一个比特。
- 如果 $f_z > f_{zi+1}$ ，将不含秘密信息的 $2i+1$ 转为 $2i$ ，否则使用标准的 LSB 嵌入。

3.2.1 代码

```
if count < secret_length:
    # 读取当前像素的最低有效位
    lsb = pixel % 2
    embed_bit = int(secret_data[count]) # 获取当前要嵌入的比特
    # 判断  $f_z > f_{zi+1}$  进行补偿
    if hist[pixel] > hist[pixel + 1]: # 如果  $f_z > f_{zi+1}$ 
        # 如果像素是  $2i+1$ ，并且没有嵌入信息，将其改为  $2i$ 
        if lsb == 1 and embed_bit == 0:
            pixel -= 1 # 将  $2i+1$  转为  $2i$ 
        else:
            # 标准 LSB 嵌入，修改最低有效位
            pixel = (pixel & ~1) | embed_bit # 修改最低位为嵌入比特
    count += 1 # 嵌入一个比特
```

3.2.2 分析

- 肉眼观察



Figure 25 左图为原图，右图为嵌入率为 1 的抗卡尔分析隐写后的图片

- 卡尔分析：

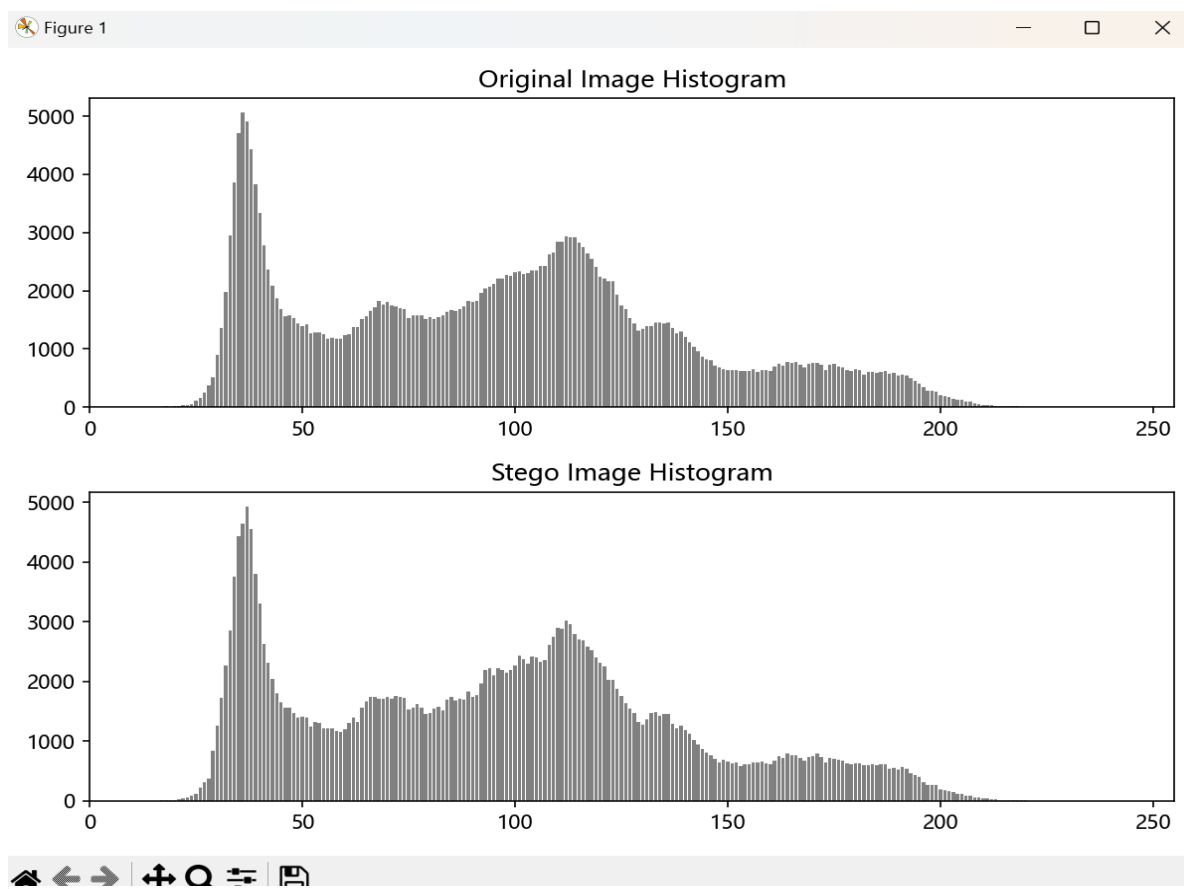


Figure 26 抗卡尔分析嵌入率为 1

- 信息量估计分析法

F1=68338, F2=67024

不含秘密信息

F1 与 F2 数值相近，无法检测到秘密信息。

● RS 分析

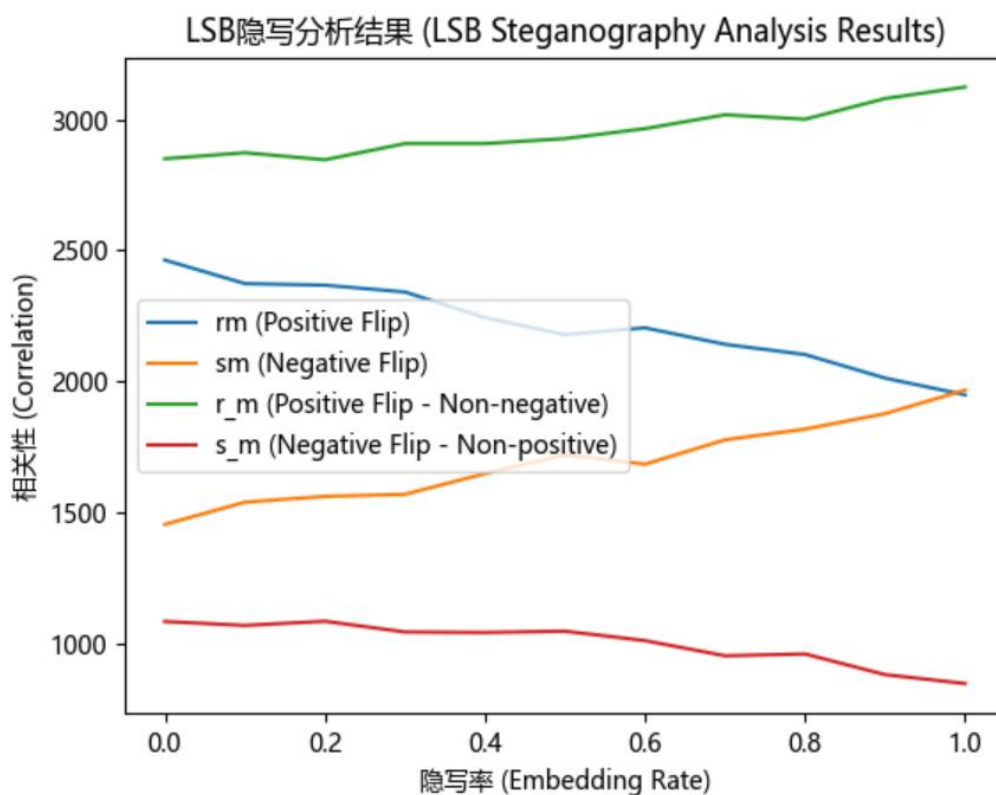


Figure 27 抗卡尔分析的 rs 分析

抗卡尔分析的图片算法仍然能被 rs 分析出来，分析出来隐写率为 20%左右。

3.3 结论

抗 RS 分析的 LSB 随机算法是如果当前像素的 LSB 为 1，则通过分析周围像素的平均值来决定是否修改当前像素的值，通过计算以当前像素为中心的 3×3 区域内的像素值总和，如果总和小于或等于当前像素值的 9 倍，则减少当前像素值；否则，增加当前像素值。

该算法在减少图像中隐写信息的可检测性方面表现出一定的效果，特别是在 RS 分析中，它能够将 RS 值降低到 0（或负数），这可能使得隐写信息更难以被检测。然而，卡方分析和信息量估计法仍然能够检测到隐写痕迹，这表明该算法可能需要进一步的改进，或者需要与其他隐写技术结合使用，以提高其在所有类型的隐写分析下的隐蔽性。

抗卡尔分析和信息量估计法分析的 LSB 随机算法算法是通过引入了 fz 和 fz_{i+1} 的补偿方法，如果像素值为 $2i+1$ 且不含秘密信息时，将其修改为 $2i$ 。这有助于减小卡方分析 (Chi-Square) 和信息量估计 (Information Entropy) 分析中的统计偏差，增

强隐写的隐蔽性。

结论

抗 RS 分析的 LSB 随机算法是如果当前像素的 LSB 为 1，则通过分析周围像素的平均值来决定是否修改当前像素的值，通过计算以当前像素为中心的 3×3 区域内的像素值总和，如果总和小于或等于当前像素值的 9 倍，则减少当前像素值；否则，增加当前像素值。

该算法在减少图像中隐写信息的可检测性方面表现出一定的效果，特别是在 RS 分析中，它能够将 RS 值降低到 0（或负数），这可能使得隐写信息更难以被检测。然而，卡方分析和信息量估计法仍然能够检测到隐写痕迹，这表明该算法可能需要进一步的改进，或者需要与其他隐写技术结合使用，以提高其在所有类型的隐写分析下的隐蔽性。

抗卡尔分析和信息量估计法分析的 LSB 随机算法算法是通过引入了 f_z 和 f_{zi+1} 的补偿方法，如果像素值为 $2i+1$ 且不含秘密信息时，将其修改为 $2i$ 。这有助于减小卡方分析（Chi-Square）和信息量估计（Information Entropy）分析中的统计偏差，增强隐写的隐蔽性。

参考文献

- [1] https://blog.csdn.net/qg_38154820/article/details/122694645
https://blog.csdn.net/weixin_46447549/article/details/121626375 Martin, G. Control of electronic resources in Australia. In Pattle, L. W., & Cox, B. J. (Eds.), Electronic resources: selection and bibliographic control. New York: The Haworth Press, 1966: 85-96.
- [2] Wang, W. L. Preparation and performance study of fluorescent carbon nanomaterials [D]. Shanghai: Shanghai Normal University, 2019.
- [3] Westfeld, A. (2001). F5 — A steganographic algorithm: High capacity despite better detection. In International Workshop on Information Hiding, 2001, 289-302.
- [4] Zhang, Y., & Wang, X. (2018). A survey of image steganography techniques. International Journal of Computer Applications, 182(30), 30-37.
- [5] C. H. S. (2014). Practical approaches to hiding information in digital images. IEEE Transactions on Information Forensics and Security, 9(1), 11-25.
- [6] Petitcolas, F. A. P., Anderson, R., & Kuhn, M. G. (1999). Information hiding—A survey. Proceedings of the IEEE, 87(7), 1062-1078.

附录